

Multiple Traffic Light Controller with Cross Walk and an Emergency Buzzer with a SMART Sensor

CEIS114 Final Course Project

May 2024

Barbara Espericueta

Prof. Alireza Kavianpour

Introduction

Multiple Traffic Light Controller with Cross Walk and an Emergency Buzzer with a SMART Sensor:

- Overall Project: Creating a Multi-Intersection Traffic Light System using Wokwi Lab, exploring the intersection of electronics, microcontroller programming, and traffic engineering. Gaining insight into the principles behind traffic light signal control and practical skills that can contribute to more efficient and sustainable urban transportation systems.
- Objective: To design and develop a two-way traffic light controller with a pedestrian crossing and an emergency signal.
- Tools: *Wokwi Lab, Arduino IDE, IoT MQTT Panel, Microsoft Word, PowerPoint, and Wix.*

CEIS 114

Module 2

Project Plan for IoT Traffic Controller

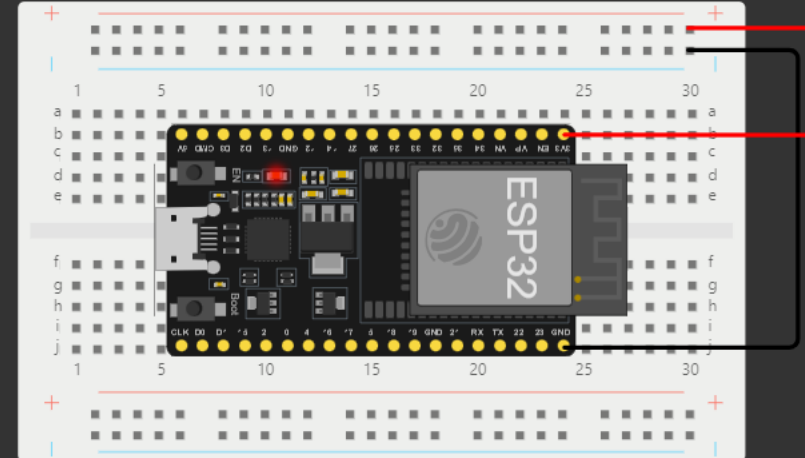
In this module, we familiarized ourselves with the two-way traffic light controller and set up our ESP32 microcontroller board in the Wokwi virtual environment.

sketch.ino • diagram.json Library Manager

```
1  /* Barbara Espericueta */
2
3  #include "WiFi.h"
4
5  void setup() {
6      Serial.begin(115200);
7      Serial.println("Initializing WiFi...");
8      WiFi.mode(WIFI_STA);
9      Serial.println("Setup done!");
10 }
11
12 void loop() {
13     Serial.println("Scanning...");
14
15     // WiFi.scanNetworks will return the number of networks found
16     int n = WiFi.scanNetworks();
17     Serial.println("Scan done!");
18     if (n == 0) {
19         Serial.println("No networks found.");
20     } else {
21         Serial.println();
22         Serial.print(n);
23         Serial.println(" networks found");
24         for (int i = 0; i < n; ++i) {
25             // Print SSID and RSSI for each network found
26             Serial.print(i + 1);
27             Serial.print(": ");
28             Serial.print(WiFi.SSID(i));
29             Serial.print(" RSSI:");
30             Serial.print(WiFi.RSSI(i));
31             Serial.print(" dBm\n");
32         }
33     }
34 }
```

Simulation

00:01.883 90%



```
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
```

ESP32 (Screenshot)

Microcontroller mounted and powered ON

ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)

configsip: 0, SPIWP:0xee

clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00

mode:DIO, clock div:2

load:0x3fff0030,len:1156

load:0x40078000,len:11456

ho 0 tail 12 room 4

load:0x40080400,len:2972

entry 0x400805dc

Initializing WiFi...

Setup done!

Scanning...

Scan done!

1 networks found

1: Wokwi-GUEST (-97)

ESP32 Wi-Fi Scan

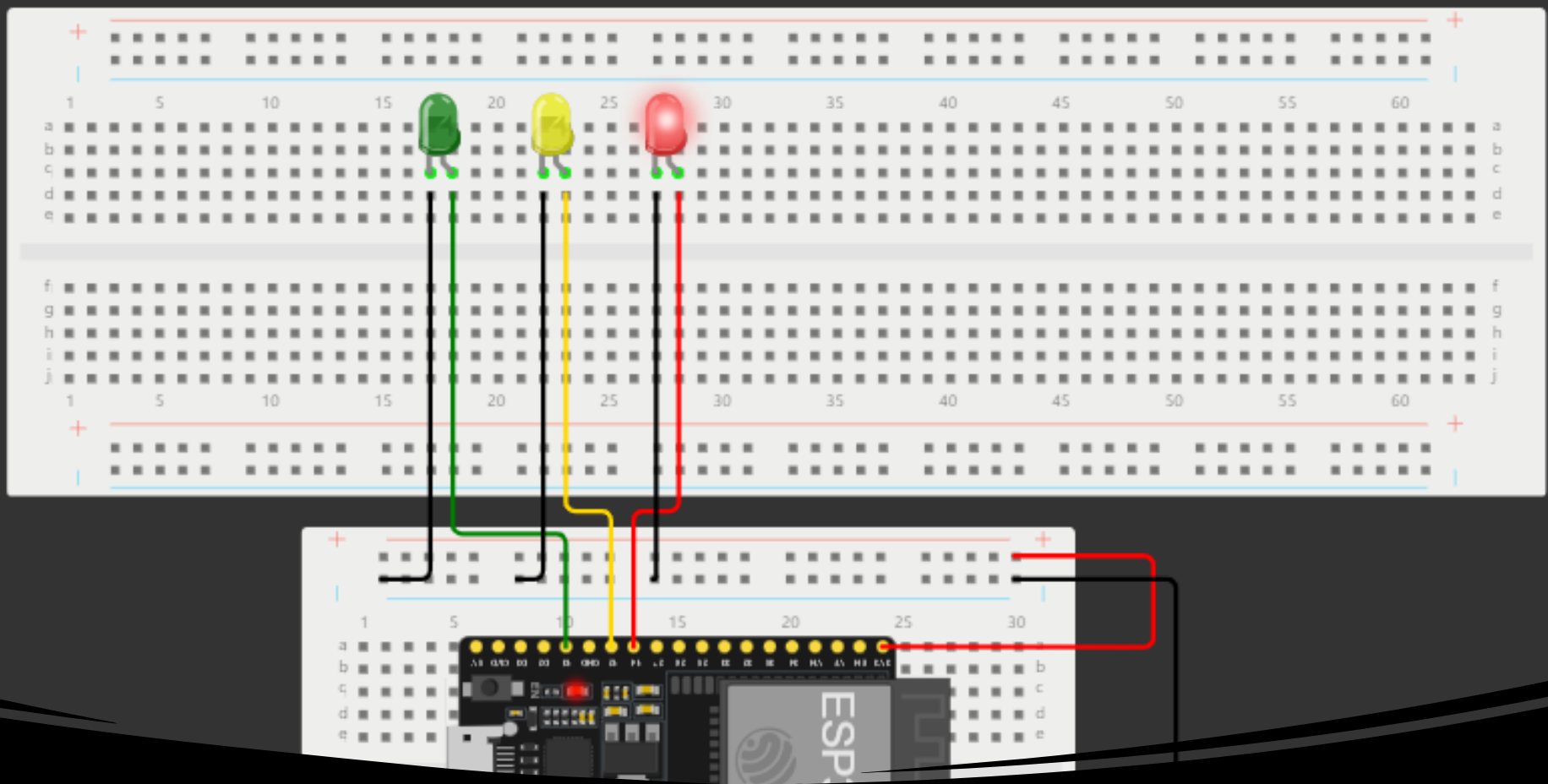
Screenshot of **Serial Monitor** showing the available networks

CEIS 114

Module 3

Creating the Traffic Controller

In this module, a program is executed to simulate a traffic light. In this step, we controlled a single LED light and then learned to control three LEDs to stimulate a single traffic light.



Picture of circuit with working LEDs

- ESP 32 Board
- Colored LEDs: Red, Yellow and Green
- 220 Ohm Resistors (optional)
- Wires
- Breadboard

Screenshot of code in Arduino IDE

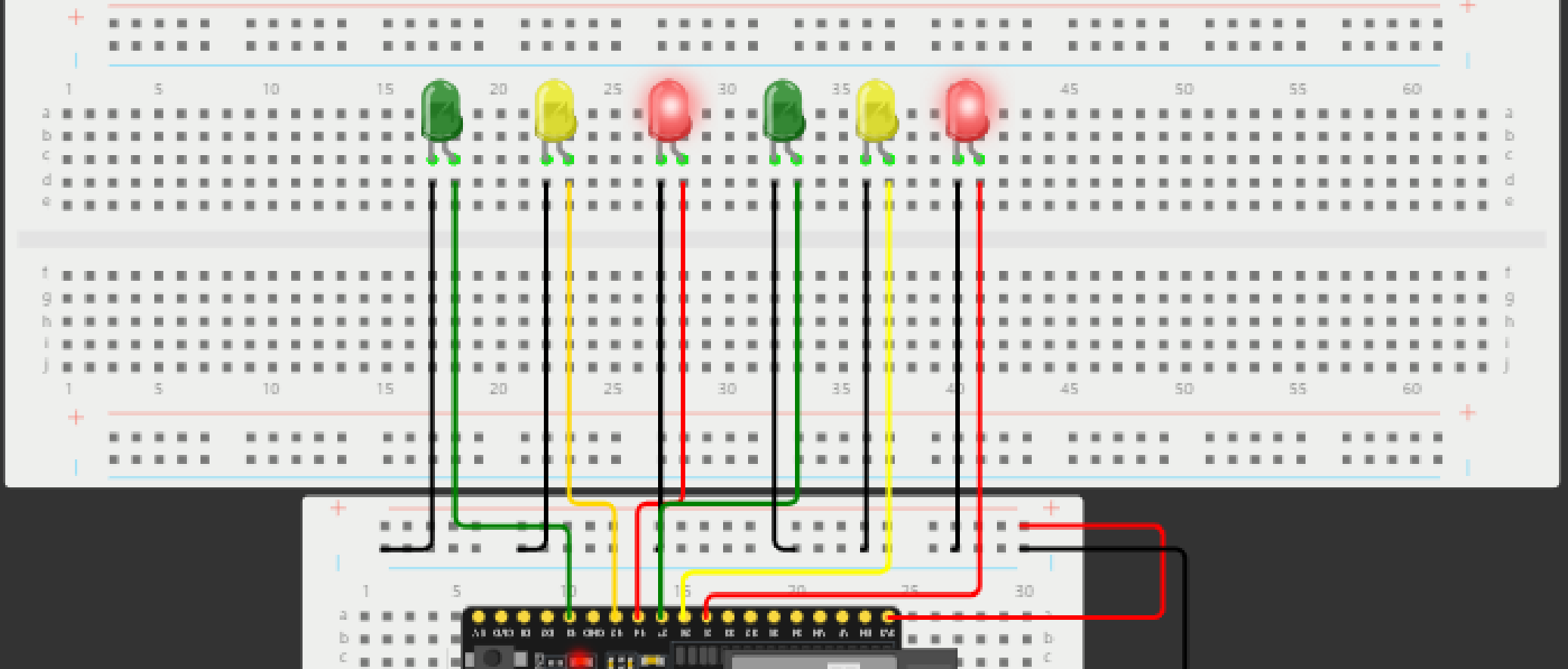
```
WOKWI SAVE SHARE Module 3 - Barbara Espericueta
sketch.ino diagram.json Library Manager
1 // === Barbara Espericueta ===
2 // Module #3 project
3
4 const int red_LED1 = 14; // The red LED1 is wired to ESP32 board pin GPIO14
5 const int yellow_LED1 = 12; // The yellow LED1 is wired to ESP32 board pin GPIO12
6 const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13
7
8 // the setup function runs once when you press reset or power the board
9 void setup() {
10 | pinMode(red_LED1, OUTPUT); // initialize digital pin GPIO14 (Red LED1) as an output.
11 | pinMode(yellow_LED1, OUTPUT); // initialize digital pin GPIO12 (yellow LED1) as an output.
12 | pinMode(green_LED1, OUTPUT); // initialize digital pin GPIO13 (green LED1) as an output.
13 | }
14
15 // the loop function runs over and over again forever
16 void loop() {
17 | // The next three lines of code turn on the red LED1
18 | digitalWrite(red_LED1, HIGH); // This should turn on the RED LED1
19 | digitalWrite(yellow_LED1, LOW); // This should turn off the YELLOW LED1
20 | digitalWrite(green_LED1, LOW); // This should turn off the GREEN LED1
21 |
22 | delay(2000); // wait for 2 seconds
23 |
24 | // The next three lines of code turn on the green LED1
25 | digitalWrite(red_LED1, LOW); // This should turn off the RED LED1
26 | digitalWrite(yellow_LED1, LOW); // This should turn off the YELLOW LED1
27 | digitalWrite(green_LED1, HIGH); // This should turn on the GREEN LED1
28 |
29 | delay(2000); // wait for 2 seconds
30 |
31 | // The next three lines of code turn on the yellow LED1
32 | digitalWrite(red_LED1, LOW); // This should turn off the RED LED1
33 | digitalWrite(yellow_LED1, HIGH); // This should turn on the YELLOW LED1
34 | digitalWrite(green_LED1, LOW); // This should turn off the GREEN LED1
35 |
36 | delay(2000); // wait for 2 seconds
37 | }
38
```


CEIS 114

Module 4

Creating a Multiple Traffic Light Controller

In this phase of the module, we implemented a program for simulating a multi-intersection traffic light controller. Encompassing a total of two traffic light systems.



Picture of circuit
with working LEDs

- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)
- Wires
- Breadboard

clk drv:0x00,q drv:0x00,d drv:0x00,cs0 drv:0x00,hd drv:0x00,wp drv:0x00

Screenshot of code in Wokwi

```
WOKWI  SAVE  SHARE  Module 4 - Barbara Espericueta 
```

```
sketch.ino ● diagram.json ● Library Manager ▼
```

```
1 // === Barbara Espericueta ===
2 // Module #4 project
3
4 // Define some labels
5 const int red_LED1 = 14; // The red LED1 is wired to ESP32 board pin GPIO14
6 const int yellow_LED1 = 12; // The yellow LED1 is wired to ESP32 board pin GPIO12
7 const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13
8 const int red_LED2 = 25; // The red LED2 is wired to Mega board pin GPIO25
9 const int yellow_LED2 = 26; // The yellow LED2 is wired to Mega board pin GPIO 26
10 const int green_LED2 = 27; // The green LED2 is wired to Mega board pin GPIO 27
11
12 // the setup function runs once when you press reset or power the board
13 void setup() {
14   pinMode(red_LED1, OUTPUT); // initialize digital pin GPIO14 (Red LED1) as an output.
15   pinMode(yellow_LED1, OUTPUT); // initialize digital pin GPIO12 (yellow LED1) as an output.
16   pinMode(green_LED1, OUTPUT); // initialize digital pin GPIO13 (green LED1) as an output.
17   pinMode(red_LED2, OUTPUT); // initialize digital pin GPIO25 (Red LED2) as an output.
18   pinMode(yellow_LED2, OUTPUT); // initialize digital pin GPIO26 (yellow LED2) as an output.
19   pinMode(green_LED2, OUTPUT); // initialize digital pin GPIO27 (green LED2) as an output.
20 }
21
22 // the loop function runs over and over again forever
23 void loop() {
24
25   // The next three lines of code turn on the red LED1
26   digitalWrite(red_LED1, HIGH); // This should turn on the RED LED1
27   digitalWrite(yellow_LED1, LOW); // This should turn off the YELLOW LED1
28   digitalWrite(green_LED1, LOW); // This should turn off the GREEN LED1
29
30   delay(1000); //Extended time for Red light#1 before the Green of the other side turns ON
31
32   // The next three lines of code turn on the green LED2 for 2 seconds
33   digitalWrite(red_LED2, LOW); // This should turn off the RED LED2
34   digitalWrite(yellow_LED2, LOW); // This should turn off the YELLOW LED2
35   digitalWrite(green_LED2, HIGH); // This should turn on the GREEN LED2
36
37   delay(2000); // wait for 2 seconds
38 }
```

CEIS 114

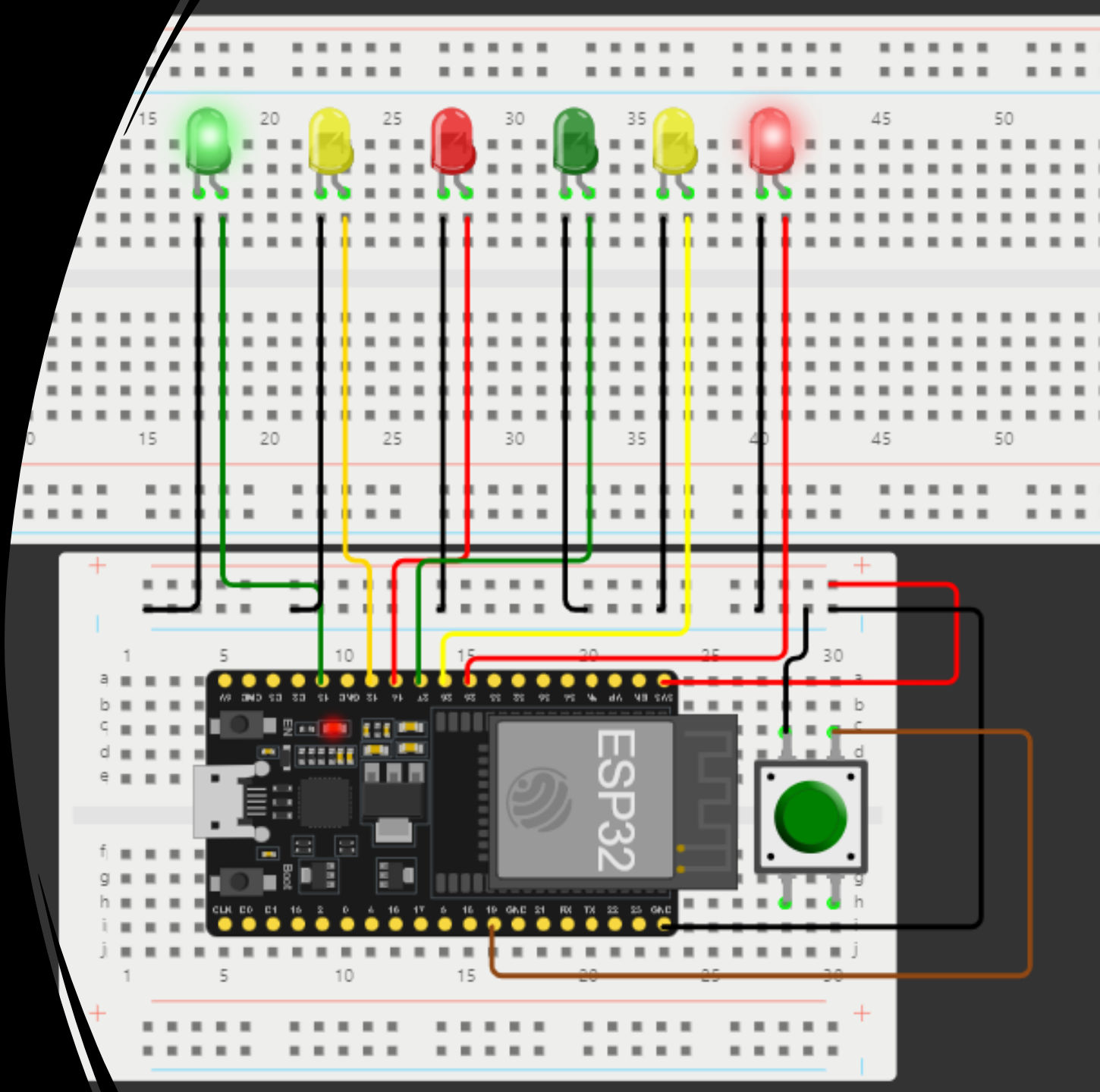
Module 5

Creating a Multiple Traffic Light Controller with a Cross Walk

In this module, we embarked on the creation of a smart traffic light system with a crosswalk for people to walk safely. To make sure everyone stays safe and moves at the right time we created a sophisticated multiple traffic light controller integrated with a pedestrian crosswalk.

Screenshot of circuit with working LEDs

- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)
- 220 Ohm Resistors (optional)
- Push Button
- Wires
- Breadboard



Screenshot of code in Wokwi

```
SAVE SHARE Module 5 - Barbara Espericueta
diagram.json Library Manager
// === Barbara Espericueta ===

// Module #5 project

const int red_LED1 = 14; // The red LED1 is wired to ESP32 board pin GPIO14
const int yellow_LED1 = 12; // The yellow LED1 is wired to ESP32 board pin GPIO12

const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13
const int red_LED2 = 25; // The red LED2 is wired to Mega board pin GPIO25
const int yellow_LED2 = 26; // The yellow LED2 is wired to Mega board pin GPIO 26
const int green_LED2 = 27; // The green LED2 is wired to Mega board pin GPIO 27

int Xw_value;

const int Xw_button = 19; //Cross Walk button

// the setup function runs once when you press reset or power the board
void setup() {
  pinMode(Xw_button, INPUT_PULLUP); // 0=pressed, 1 = unpressed button
  Serial.begin(115200);
  pinMode(red_LED1, OUTPUT); // initialize digital pin 14 (Red LED1) as an output.
  pinMode(yellow_LED1, OUTPUT); // initialize digital pin 12 (yellow LED1) as an output.
  pinMode(green_LED1, OUTPUT); // initialize digital pin 13 (green LED1) as an output.

  pinMode(red_LED2, OUTPUT); // initialize digital pin 25(Red LED2) as an output.
  pinMode(yellow_LED2, OUTPUT); // initialize digital pin 26 (yellow LED2) as an output.
  pinMode(green_LED2, OUTPUT); // initialize digital pin 27 (green LED2) as an output.
}

// the loop function runs over and over again forever
void loop() {
  // read the cross walk button value:
  Xw_value=digitalRead(Xw_button);

  if (Xw_value == LOW ){ // if crosswalk button (X-button) pressed
```

Screenshot of Serial Monitor in Wokwi

Screenshot of output in Serial Monitor

1 is wired to ESP
is wired to ESP32
wired to Mega bo
2 is wired to Meg
is wired to Mega

s reset or power

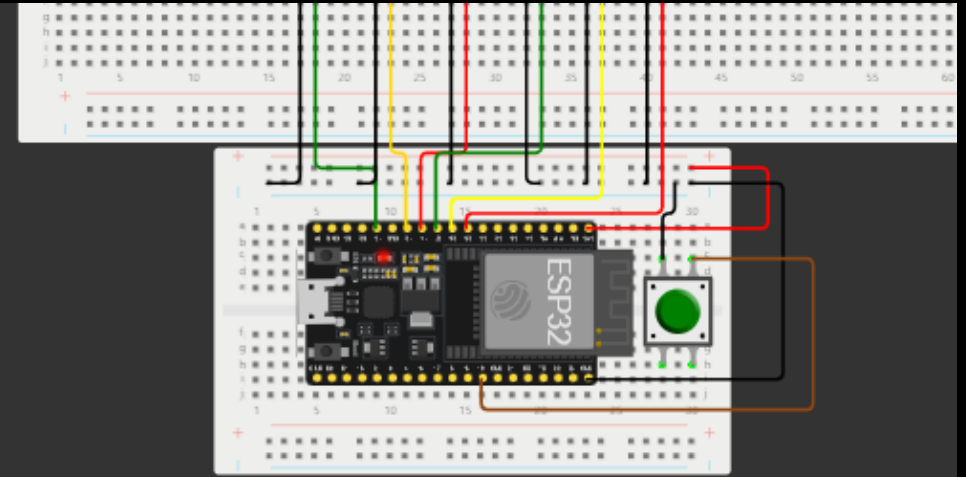
d, 1 = unpressed

tal pin 14 (Red L
igital pin 12 (ye
igital pin 13 (gr

tal pin 25(Red LE
igital pin 26 (ye
igital pin 27 (gre

forever

(X-button) press



```
== Do Not Walk ==  
== Do Not Walk ==  
Count = 10 == Walk ==  
Count = 9 == Walk ==  
Count = 8 == Walk ==  
Count = 7 == Walk ==  
Count = 6 == Walk ==  
Count = 5 == Walk ==  
Count = 4 == Walk ==  
Count = 3 == Walk ==  
Count = 2 == Walk ==  
Count = 1 == Walk ==  
== Do Not Walk ==
```

CEIS 114

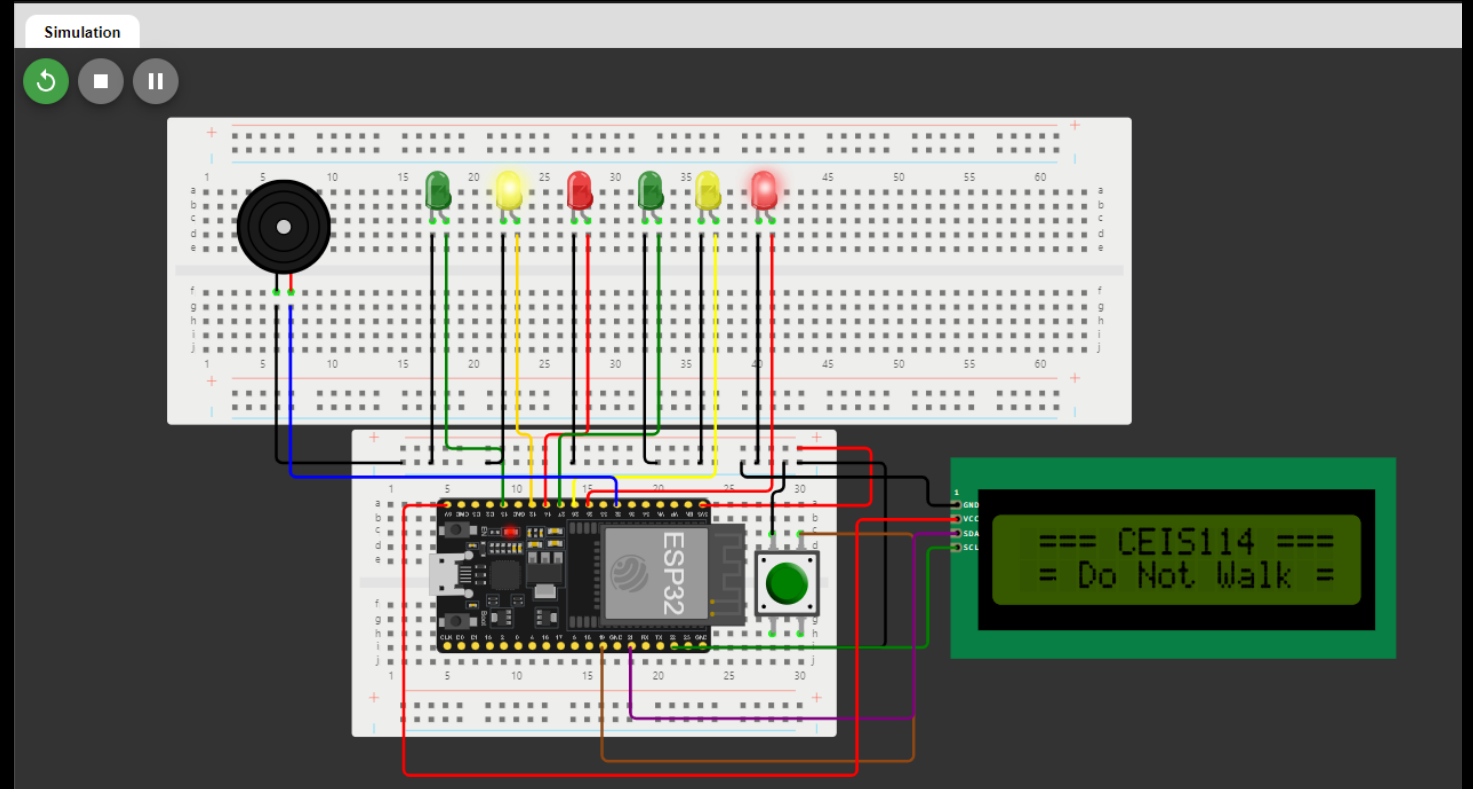
Module 6

Creating a Multiple Traffic Light Controller with a Cross Walk and an Emergency Buzzer

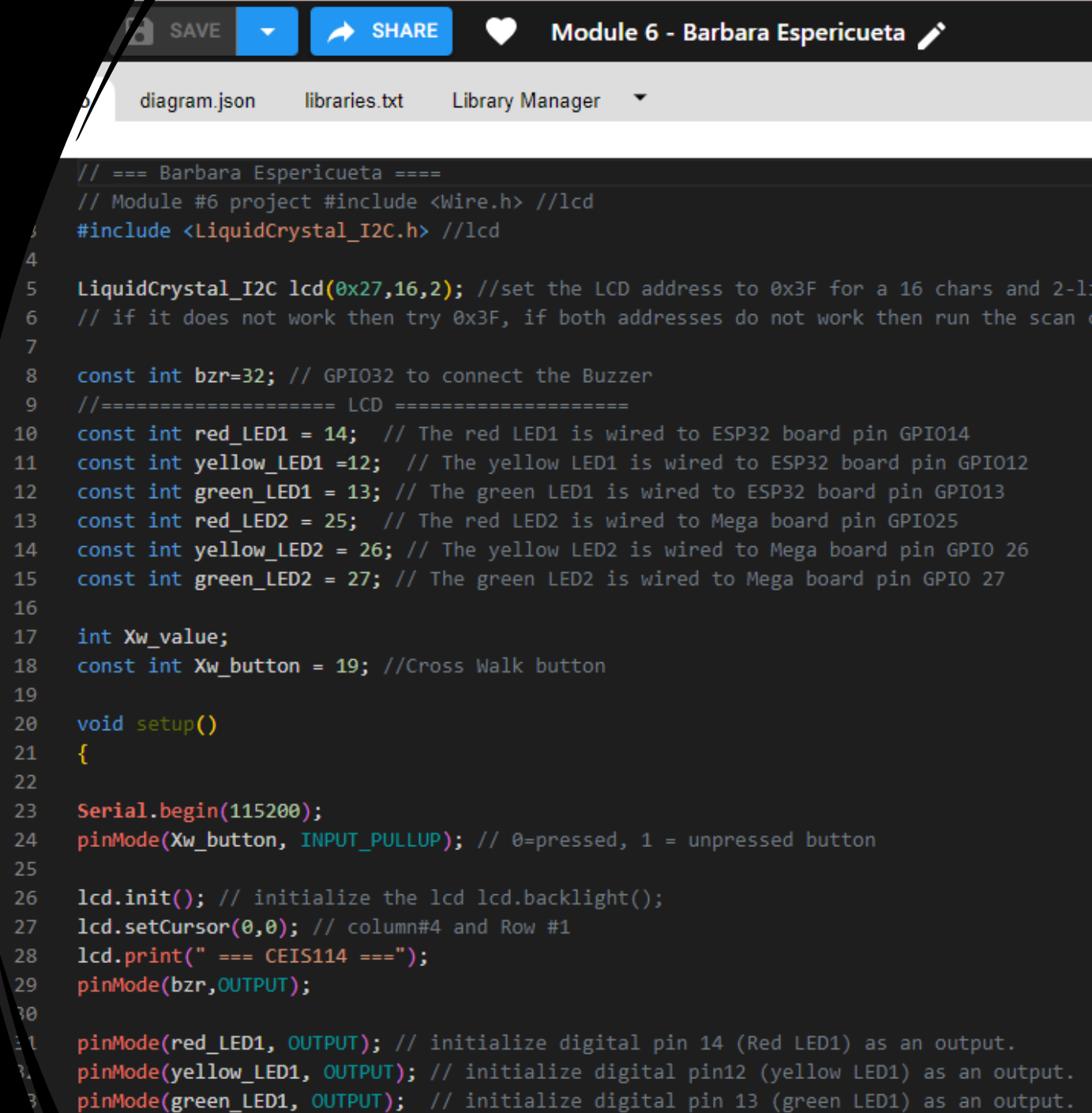
In this phase of the module, we embarked on the creation of a smart traffic light system with a crosswalk of a display for people to see the instructions, and a buzzer to sound to alert them when to walk and not to walk at the intersection.

Picture of the circuit with working LEDs and LCD display

- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)
- 220 Ohm Resistors (optional)
- Push Button
- LCD Unit with Message Display
- Wires
- Breadboard



Screenshot of code in Code Editor

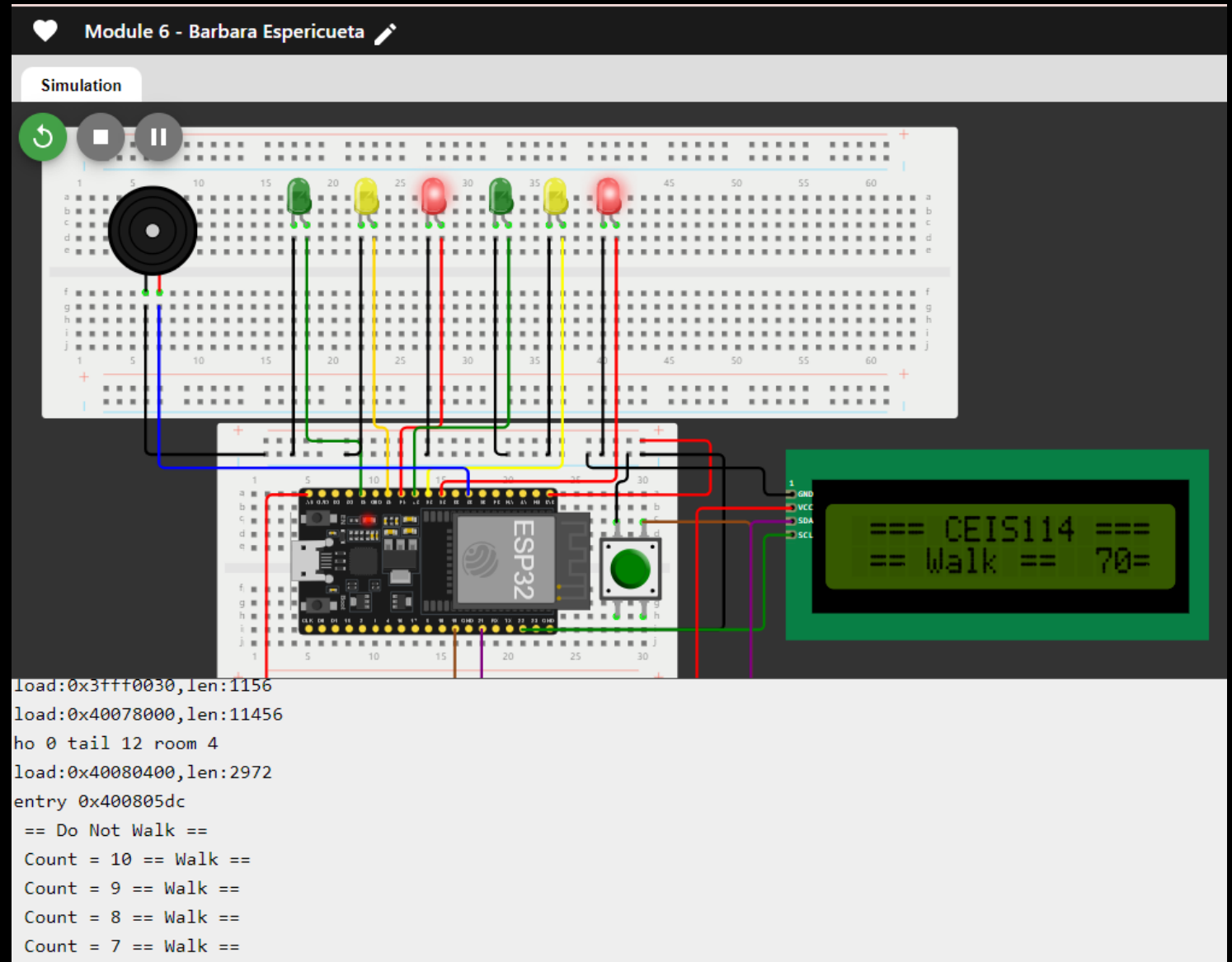


The screenshot shows an IDE window with a dark theme. At the top, there are buttons for 'SAVE' and 'SHARE', a heart icon, and the text 'Module 6 - Barbara Espericueta'. Below this is a tab bar with 'diagram.json', 'libraries.txt', and 'Library Manager'. The main area displays C++ code for an Arduino project. The code includes comments in Spanish and English, and uses the `LiquidCrystal_I2C` library. It defines pin constants for LEDs and a buzzer, and includes a `setup()` function that initializes the LCD and pins.

```
// === Barbara Espericueta ===  
// Module #6 project #include <Wire.h> //lcd  
#include <LiquidCrystal_I2C.h> //lcd  
  
LiquidCrystal_I2C lcd(0x27,16,2); //set the LCD address to 0x3F for a 16 chars and 2-1  
// if it does not work then try 0x3F, if both addresses do not work then run the scan  
  
const int bzt=32; // GPIO32 to connect the Buzzer  
//===== LCD =====  
const int red_LED1 = 14; // The red LED1 is wired to ESP32 board pin GPIO14  
const int yellow_LED1 =12; // The yellow LED1 is wired to ESP32 board pin GPIO12  
const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13  
const int red_LED2 = 25; // The red LED2 is wired to Mega board pin GPIO25  
const int yellow_LED2 = 26; // The yellow LED2 is wired to Mega board pin GPIO 26  
const int green_LED2 = 27; // The green LED2 is wired to Mega board pin GPIO 27  
  
int Xw_value;  
const int Xw_button = 19; //Cross Walk button  
  
void setup()  
{  
  
  Serial.begin(115200);  
  pinMode(Xw_button, INPUT_PULLUP); // 0=pressed, 1 = unpressed button  
  
  lcd.init(); // initialize the lcd lcd.backlight();  
  lcd.setCursor(0,0); // column#4 and Row #1  
  lcd.print(" === CEIS114 ===");  
  pinMode(bzt,OUTPUT);  
  
  pinMode(red_LED1, OUTPUT); // initialize digital pin 14 (Red LED1) as an output.  
  pinMode(yellow_LED1, OUTPUT); // initialize digital pin12 (yellow LED1) as an output.  
  pinMode(green_LED1, OUTPUT); // initialize digital pin 13 (green LED1) as an output.
```

Screenshot of Serial Monitor

Screenshot of output in Serial Monitor

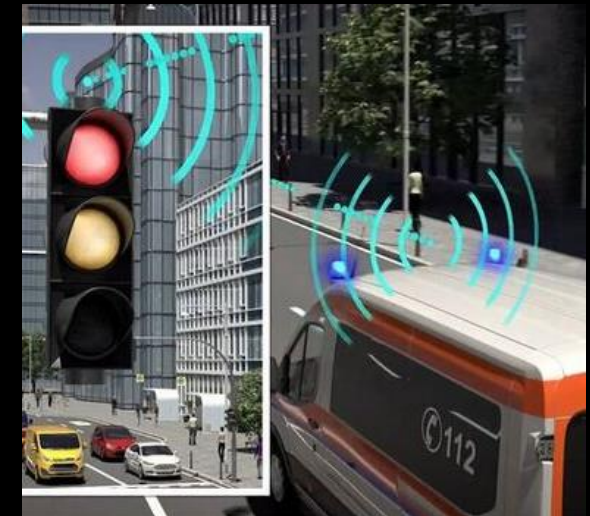
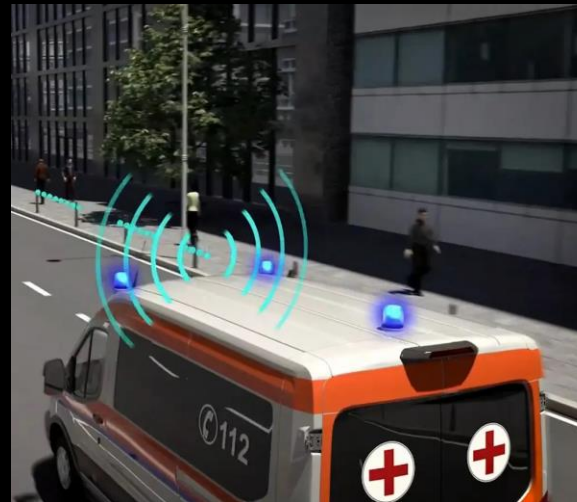
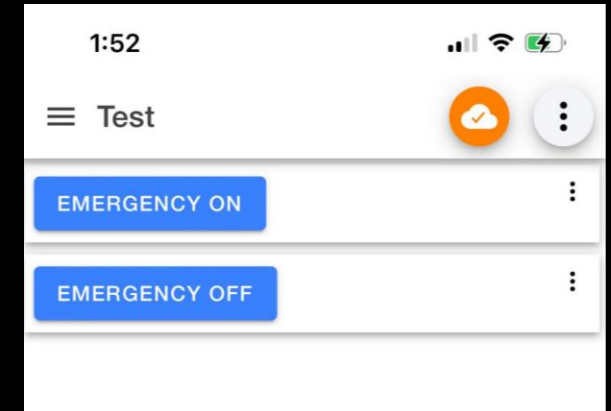
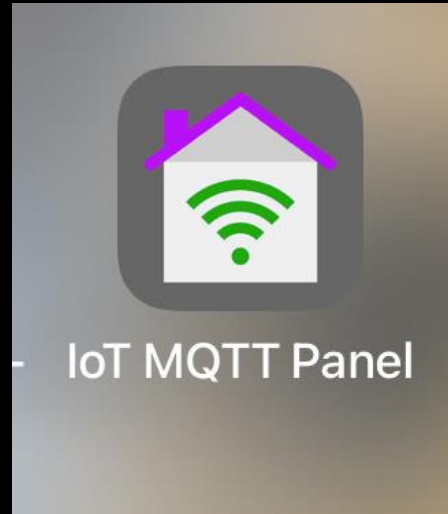


CEIS 114

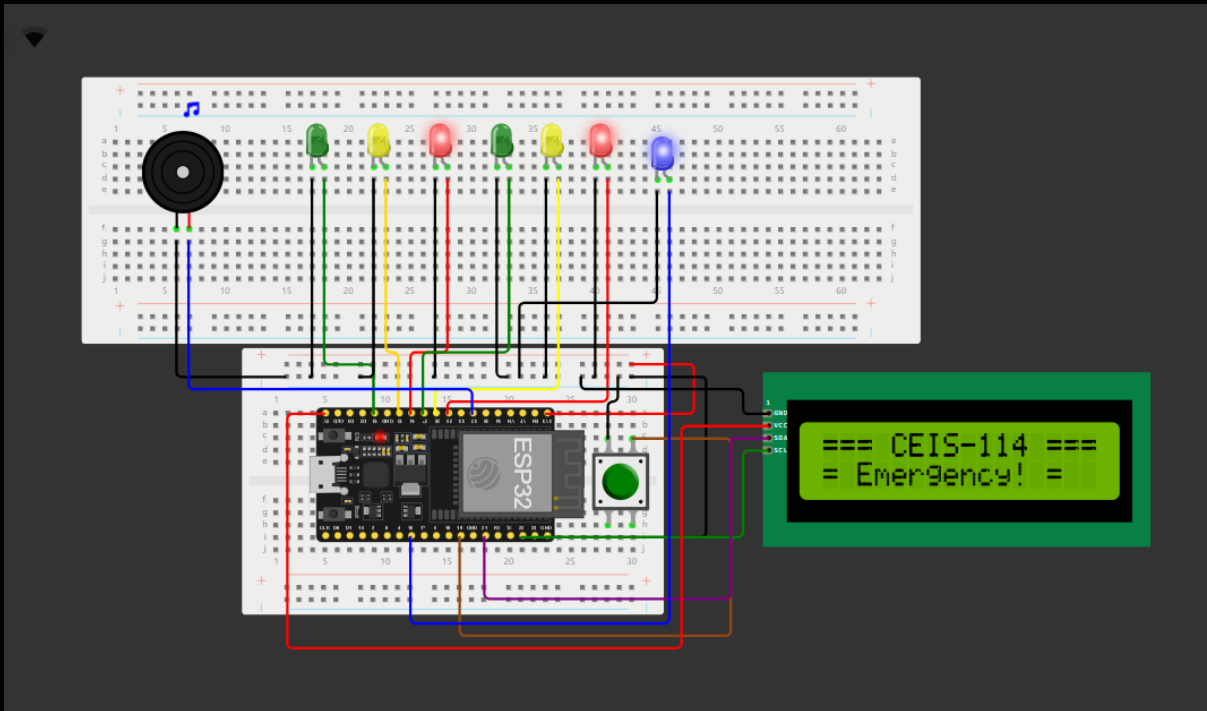
Week 7 Project

Creating a Multiple Traffic Light Controller with a Cross Walk and an Emergency Buzzer with secured IoT Control via Web

In this final step of our multi-intersection traffic light system, we entered the world of connected devices and created our own simple IoT system. We used the MQTT protocol, which is an easy-to-understand language that devices use to talk to each other.

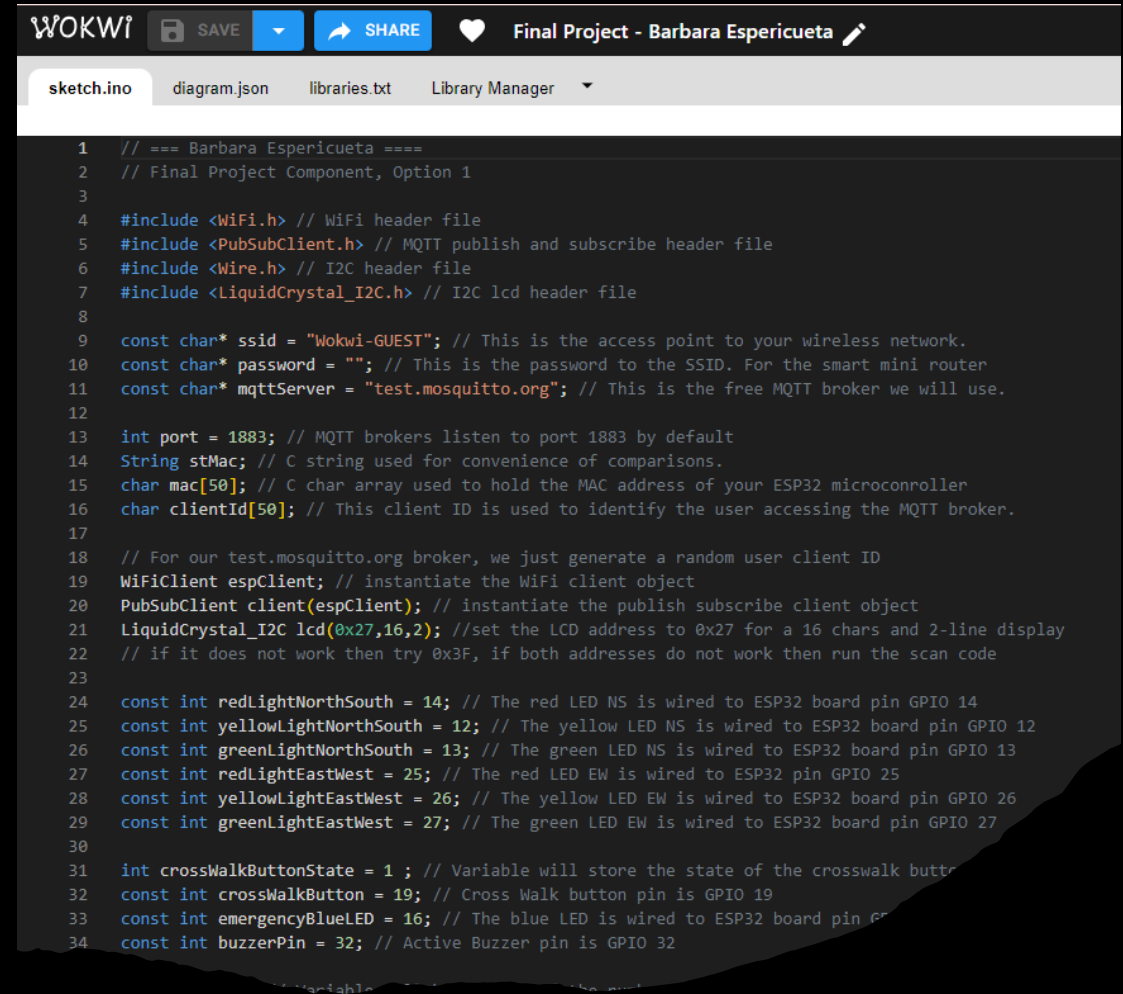


Screenshot of circuit **with working LEDs and LCD display** (Building/Operation)



- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)
- One Blue LED – Emergency Light
- Push Button
- LCD Unit
- Buzzer
- Wires
- Breadboard

Screenshot of code in Code Editor (Testing)



```
1 // === Barbara Espericueta ===
2 // Final Project Component, Option 1
3
4 #include <WiFi.h> // WiFi header file
5 #include <PubSubClient.h> // MQTT publish and subscribe header file
6 #include <Wire.h> // I2C header file
7 #include <LiquidCrystal_I2C.h> // I2C lcd header file
8
9 const char* ssid = "Wokwi-GUEST"; // This is the access point to your wireless network.
10 const char* password = ""; // This is the password to the SSID. For the smart mini router
11 const char* mqttServer = "test.mosquitto.org"; // This is the free MQTT broker we will use.
12
13 int port = 1883; // MQTT brokers listen to port 1883 by default
14 String stMac; // C string used for convenience of comparisons.
15 char mac[50]; // C char array used to hold the MAC address of your ESP32 microconroller
16 char clientId[50]; // This client ID is used to identify the user accessing the MQTT broker.
17
18 // For our test.mosquitto.org broker, we just generate a random user client ID
19 WiFiClient espClient; // instantiate the WiFi client object
20 PubSubClient client(espClient); // instantiate the publish subscribe client object
21 LiquidCrystal_I2C lcd(0x27,16,2); //set the LCD address to 0x27 for a 16 chars and 2-line display
22 // if it does not work then try 0x3F, if both addresses do not work then run the scan code
23
24 const int redLightNorthSouth = 14; // The red LED NS is wired to ESP32 board pin GPIO 14
25 const int yellowLightNorthSouth = 12; // The yellow LED NS is wired to ESP32 board pin GPIO 12
26 const int greenLightNorthSouth = 13; // The green LED NS is wired to ESP32 board pin GPIO 13
27 const int redLightEastWest = 25; // The red LED EW is wired to ESP32 pin GPIO 25
28 const int yellowLightEastWest = 26; // The yellow LED EW is wired to ESP32 board pin GPIO 26
29 const int greenLightEastWest = 27; // The green LED EW is wired to ESP32 board pin GPIO 27
30
31 int crossWalkButtonState = 1 ; // Variable will store the state of the crosswalk button
32 const int crossWalkButton = 19; // Cross Walk button pin is GPIO 19
33 const int emergencyBlueLED = 16; // The blue LED is wired to ESP32 board pin GPIO 16
34 const int buzzerPin = 32; // Active Buzzer pin is GPIO 32
```

```

10 char* password = "test"; //
11 char* mqttServer = "test"
12
13 port = 1883; // MQTT broker
14 hostMac; // C string used
15 char mac[50]; // C char array
16 char clientId[50]; // This cli
17
18 char* url = "http://test.mosquitto.org"
19 MQTTClient espClient; // insta
20 MQTTClient client(espClient)
21 Crystal_I2C lcd(0x27,16,
22 it does not work then tr
23
24 int redLightNorthSouth =
25 int yellowLightNorthSouth
26 int greenLightNorthSouth
27 int redLightEastWest = 2
28 int yellowLightEastWest
29 int greenLightEastWest =
30
31 int crossWalkButtonState = 1;
32 int crossWalkButton = 19

```

== Do Not Walk ==

== Do Not Walk ==

== Do Not Walk ==

== Walk == 15

== Walk == 14

== Walk == 13

== Walk == 12

= Emergency! =

= Emergency! =

= Emergency! =

= Emergency! =

== Do Not Walk ==

== Do Not Walk ==

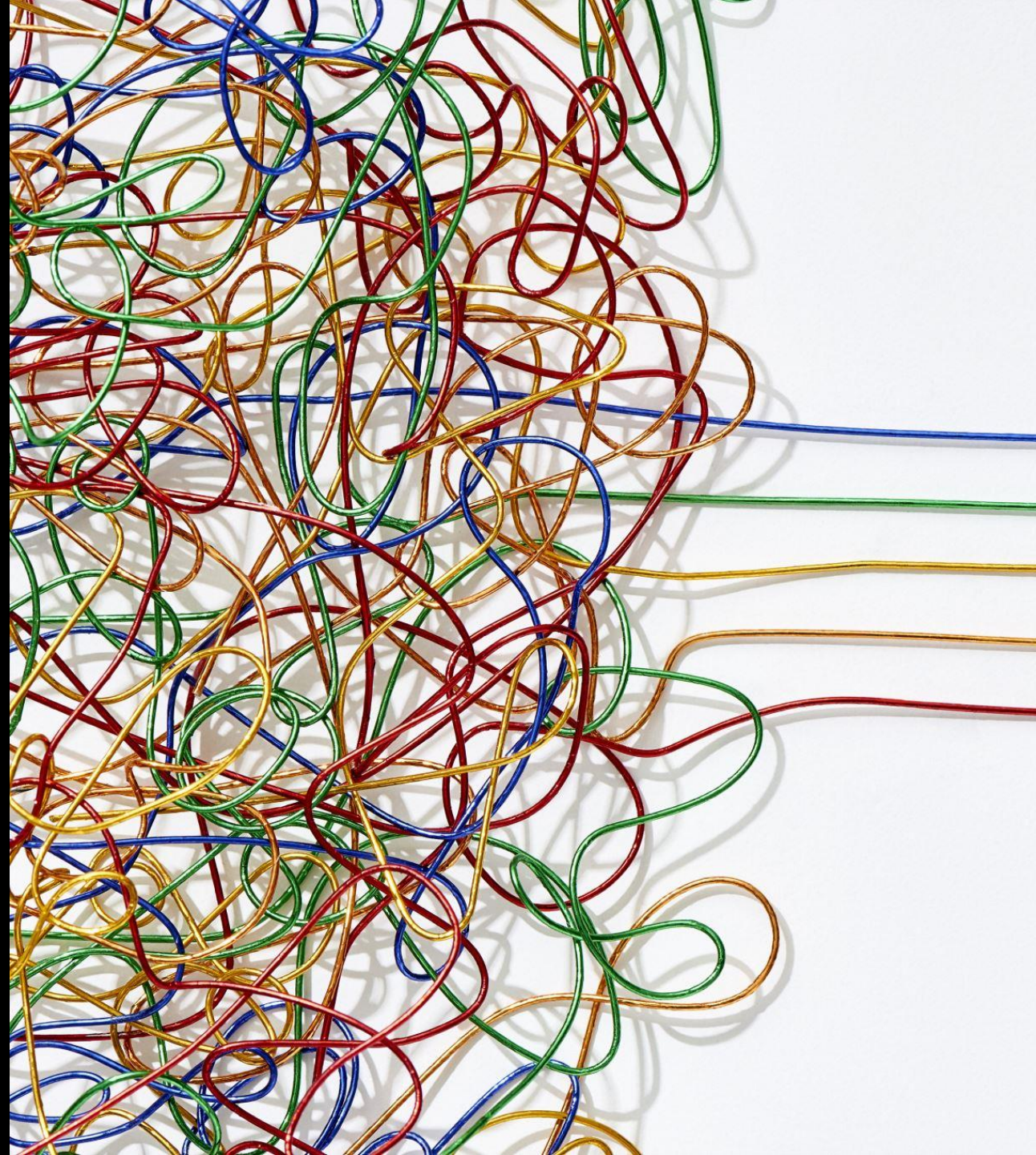
== Do Not Walk ==

Screenshot of Serial Monitor (Testing)

Screenshot of output in Serial Monitor

Challenges/Lessons Learned

A challenge I faced during this project was the wiring between the display screen and the ESP32. After looking at a data sheet on the ESP32, I found that it was easier to understand what goes where, by referencing through GPI PIN Numbers. Then understood that 8 pins could be used to allow communication, 16 PWD for analog outputs, 2 digital-analog converters, 2 I2S interfaces, and 10 Capacitive sensing GPIOs. Then I was able to identify that the display screen had two connections for communication, one for voltage, and a ground. Lessons learned in this project, completing a project gives you more knowledge than just reading or watching.



Career Skills

Skills I developed that will benefit my career would be getting familiar with basic electronics: circuits, sensors, and programming microcontrollers. Programming is a skill I sharpened, gaining proficiency in the programming language in writing the software that runs the microcontroller a must-have skill in cybersecurity. Skills in Networking is another skill I gained. Wi-Fi, LAN, WAN, and cellular networks are essential, devices that can connect to the internet and communicate with other devices are growing fast knowing these skills will help any organization.





Conclusion

With the exponential growth in connected devices, each item in the IoT system communicates packets of data that require reliable connectivity, storage, and security. These projects will help us deal with such challenges as monitoring and securing large volumes of data.

